

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

INFOMRE DE PRUEBAS

PROYECTO: DieselMaps

Sistema de Geolocalización de Estaciones de Combustible

Fecha: Mayo 2026

Firmado por: Líder de Calidad — Grupo 2

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

1. Información General del Proyecto	1
1.1 Módulos Evaluados	2
2. Resumen de Resultados	2
3. Resultados de Pruebas de Integración	2
3.1 Resumen de Casos de Integración (CPI)	2
4. Resultados de Pruebas de Aceptación (CPA)	10
5. Resultados de Pruebas de Sistema (CPS)	14
5.1 Defectos Identificados en Pruebas de Sistema	15
6. Pruebas de Atributos de Calidad No Funcional	15
6.1 Pruebas de Carga — Apache JMeter	15
Endpoints Críticos Identificados para Pruebas de Carga	16
6.2 Análisis de Desempeño	16
6.3 Evaluación de Escalabilidad	16
6.4 Evaluación de Disponibilidad	16
6.5 Análisis de Modificabilidad — SonarQube	17
7. Gestión de Riesgos	17
8. Recomendaciones	18
8.1 Correcciones Críticas — Prioridad 1 (Antes de Producción)	18
8.2 Mejoras de Calidad — Prioridad 2	18
8.3 Mejoras de Cobertura de Pruebas — Prioridad 3	18
8.4 Mejoras de Arquitectura — Largo Plazo	19
9. Asignación de Responsabilidades	19
10. Conclusiones	19
10.1 Estado General del Sistema	19
10.2 Logros Verificados	20
10.3 Deficiencias Identificadas	20
10.4 Veredicto Final	20

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

1. Información General del Proyecto

DieselMaps es una aplicación web de geolocalización desarrollada con arquitectura en tres capas: frontend React 19 + Vite (puerto 5175), backend Spring Boot 3.5.5 con Java 17 (puerto 8080) y base de datos MySQL 8.0 (esquema diesel_maps). Implementa autenticación JWT stateless con roles USER, OPERATOR y ADMIN, e integración con Google Maps API para visualización de estaciones de combustible.

Componente	Tecnología / Versión	URL / Puerto
Backend	Spring Boot 3.5.5 / Java 17+	http://localhost:8080
Frontend	React 19 + Vite 8 / Node.js 20	http://localhost:5175
Base de Datos	MySQL Server 8.0	localhost:3306 / diesel_maps
Pruebas API	Postman + Node.js (axios)	Colección DieselMaps API
Carga	Apache JMeter 5.6+	Plan DieselMaps.jmx
Calidad estática	SonarQube 10.x Community	http://localhost:9000

1.1 Módulos Evaluados

- Módulo de Autenticación: registro, login JWT, control de acceso por rol (RBAC)
- Módulo de Estaciones: creación, consulta haversine, detalle, actualización de precios
- Módulo de Inventario de Combustible: gestión de stock (CORRIENTE, EXTRA, DIESEL, GAS)
- Módulo de Compras: creación, descuento de stock, historial
- Módulo Administrativo: CRUD global de estaciones por ADMIN
- Módulo de Usuario: favoritos y alertas de precios
- Frontend React: rutas protegidas, Google Maps, comparador, historial Recharts, tienda
- Seguridad: Spring Security stateless, BCrypt, CORS, manejo global de excepciones
- Base de datos: MySQL + Flyway migrations (V1.0.4 a V1.0.7)

2. Resumen de Resultados

Tipo de Prueba	Total Casos	Aprobados	Parciales	Rechazados	% Éxito
Pruebas de Integración	18	18	0	0	100%
Pruebas de Aceptación	6	6	0	0	100%
Pruebas de Sistema	10	10	0	0	100%
TOTALES	34	34	0	0	100%

3. Resultados de Pruebas de Integración

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

3.1 Resumen de Casos de Integración (CPI)

Estrategia aplicada: Bottom-Up incremental desde repositorios JPA → servicios → controladores REST → frontend.

ID	Descripción	Módulos Principales	Estado
CPI-001	Registro de usuario (USER/OPERATOR)	AuthController → UserRepository → JwtUtil	APROBADO
CPI-002	Autenticación y generación de JWT	AuthService → BCrypt → JwtUtil	APROBADO
CPI-003	Acceso con token inválido/expirado	JwtFilter → SecurityConfig → GlobalExceptionHandler	APROBADO
CPI-004	Creación de estación (OPERATOR)	StationController → StationService → FuelInventoryService	APROBADO
CPI-005	Consulta estaciones cercanas (haversine)	StationController → StationRepository (haversine)	APROBADO
CPI-006	Detalle de estación (autenticado)	StationController → StationService → MySQL	APROBADO
CPI-007	Actualización de precio con historial y alertas	StationService → FuelPriceRepo → AlertRepository	APROBADO
CPI-008	Consulta inventario por estación	FuelInventoryController → FuelInventoryRepository	APROBADO
CPI-009	Ajuste de stock (OPERATOR/ADMIN)	FuelInventoryService → FuelInventoryHistoryRepository	APROBADO
CPI-010	Creación de compra de combustible	FuelPurchaseService → FuelInventory → MySQL	APROBADO
CPI-011	Historial de compras del usuario	FuelPurchaseController → FuelPurchaseRepository	APROBADO
CPI-012	Gestión administrativa de estaciones (ADMIN)	AdminStationController → StationRepository	APROBADO
CPI-013	Toggle de favoritos	UserController → UserFavoriteRepository	APROBADO
CPI-014	Consulta y marcado de alertas	UserController → AlertRepository	APROBADO
CPI-015	Integración Google Maps (rutas)	MapView/Routes → Google Maps API → DirectionsService	APROBADO
CPI-016	Historial de precios (Recharts)	StationController → PriceHistoryRepository → Frontend	APROBADO
CPI-017	Control de acceso por rol (HTTP 403)	JwtFilter → @PreAuthorize → GlobalExceptionHandler	APROBADO
CPI-018	Disponibilidad de base de datos (MySQL caído)	StationController → GlobalExceptionHandler (falla)	APROBADO

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

CASO DE INTEGRACIÓN CPI-001 — REGISTRO DE USUARIO		
ID	CPI-001	
Caso de Uso	UCC-01 Registro de nuevo usuario (USER/OPERATOR)	
Objetivo	Validar que el frontend envía datos de registro al backend, el servicio crea el usuario con contraseña BCrypt y retorna un token JWT válido, y el registro queda persistido en la tabla users de MySQL.	
Precondiciones	Backend Spring Boot activo en puerto 8080. Base de datos MySQL diesel_maps disponible. Frontend disponible en puerto 5175. El username y email a registrar no existen previamente en la BD.	
Módulos Integrados	Register.jsx → authAPI.register → POST /api/auth/register → AuthController → AuthService → BCryptPasswordEncoder → UserRepository → MySQL (tabla users) → JwtUtil → AuthResponse	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	El usuario accede a /register en el frontend	Formulario de registro renderizado correctamente
2	Diligencia username: "test_user", email: " test@dieselmaps.co ", password: "Test1234!", role: "USER", phone: "3001234567"	Campos válidos; botón Registrar habilitado
3	Hace clic en Registrar	Frontend envía POST /api/auth/register con payload JSON
4	Backend recibe request y valida con @Valid RegisterRequest	Validaciones de campo (email, no-blank) pasadas
5	AuthService verifica email no duplicado vía userRepository.existsByEmail()	Email no existe → continúa el flujo
6	Servicio hashea password con BCrypt y persiste User en MySQL	Registro exitoso en tabla users; passwordHash almacenado (no texto plano)
7	JwtUtil genera token con username y rol; AuthResponse retornado (HTTP 200)	Token JWT con claims: sub=test_user, role=USER, exp=+24h
8	Frontend almacena token en localStorage y redirige a /map	Usuario autenticado; ruta /map accessible; Navbar muestra username
Resultado Esperado: Usuario registrado exitosamente; token JWT válido recibido; persistencia en MySQL verificada; redirección automática a /map.		

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Criterio de Aceptación: HTTP 200, JSON con token no-nulo, usuario presente en tabla users con passwordHash BCrypt, redirección funcional.

Estado: APROBADO

Responsable: Jonathan cobos y Camilo Barbosa

CASO DE INTEGRACIÓN CPI-002 — AUTENTICACIÓN Y JWT

ID	CPI-002
Caso de Uso	UCC-02 Inicio de sesión con credenciales válidas
Objetivo	Verificar que el login retorna JWT válido y que el token es reconocido en endpoints protegidos.
Precondiciones	Usuario "test_user" previamente registrado en BD. Backend y BD activos.
Módulos Integrados	Login.jsx → authAPI.login → POST /api/auth/login → AuthController → AuthService → BCryptPasswordEncoder.matches → JwtUtil.generateToken → JwtFilter (requests siguientes)

DETALLE DEL FLUJO DE PRUEBA

No.	Acción / Entrada	Resultado Esperado
1	El usuario accede a /login e ingresa username y password correctos	Formulario de login visible
2	Frontend envía POST /api/auth/login con { username, password }	Request enviado correctamente
3	AuthService busca usuario y verifica hash BCrypt	Coincidencia de credenciales
4	JwtUtil.generateToken() crea token con subject y role como claim	Token generado con expiración de 24h (86400000ms según application.properties)
5	AuthResponse{ token, username, role } devuelto al frontend	HTTP 200 con JSON de respuesta
6	Frontend almacena token; hace GET /api/stations/1 con Authorization: Bearer <token>	JwtFilter extrae y valida token; request procesado
7	Intento de login con password incorrecta	HTTP 400 con mensaje "Credenciales incorrectas"

Resultado Esperado General: Token JWT generado y reconocido correctamente. Credenciales incorrectas retornan error descriptivo.

Criterio de Aceptación: HTTP 200 en login exitoso. Token decodificable con claims correctos. HTTP 400 en login fallido.

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Estado: APROBADO	Responsable: Jonathan cobos y Camilo Barbosa
-------------------------	--

CASO DE INTEGRACIÓN CPI-004 — CREACIÓN DE ESTACIÓN		
ID	CPI-004	
Caso de Uso	UCC-13 Creación de nueva estación de combustible	
Objetivo	Validar la integración completa al crear una estación: frontend → backend → persistencia en MySQL con precios e inventario inicial.	
Precondiciones	Usuario autenticado con rol OPERATOR o ADMIN. Token JWT válido en localStorage. Backend y MySQL activos.	
Módulos Integrados	CreateStation.jsx → stationsAPI.create → POST /api/stations → StationController(@PreAuthorize OPERATOR/ADMIN) → StationService → StationRepository + FuelPriceRepository + FuelInventoryService → MySQL	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Operador accede a /create-station (ruta protegida por ProtectedRoute requiredRoles)	Formulario de creación renderizado
2	Diligencia nombre, marca, latitud, longitud, dirección y precios por tipo de combustible	Campos validados en frontend
3	StationService verifica que req.getPrices() no sea null ni vacío	Excepción si faltan precios
4	POST /api/stations enviado con payload StationRequest	Backend recibe y valida con @Valid
5	StationService crea entidad Station, asigna operador por username autenticado	Station persistida en tabla stations con operator_id
6	FuelPriceRepository persiste precios por tipo de combustible (unique constraint station_id+fuel_type)	Registros en tabla fuel_prices
7	FuelInventoryService inicializa inventario por tipo	Registros en tabla fuel_inventory
8	HTTP 201 retornado con StationDTO	Frontend muestra confirmación; estación aparece en mapa
9	Intento de crear estación con precio negativo	HTTP 400 con error de validación

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

Resultado Esperado General: Estación creada y persistida correctamente con precios e inventario inicializado. Validaciones frontend y backend activas.

Criterio de Aceptación: HTTP 201, estación visible en GET /api/stations/nearby, registros en fuel_prices y fuel_inventory.

Estado: APROBADO

Responsable: Jonathan cobos y Camilo Barbosa

CASO DE INTEGRACIÓN CPI-010 — COMPRA DE COMBUSTIBLE

ID	CPI-010
Caso de Uso	UCC-18 Compra de combustible en estación
Objetivo	Verificar que la compra descuenta correctamente el stock del inventario, persiste la compra con detalles y retorna el resumen al usuario.
Precondiciones	Usuario autenticado (cualquier rol). Estación con inventario DIESEL $\geq$ 50L disponible. Backend y MySQL activos.
Módulos Integrados	Store.jsx $\rightarrow$ fuelPurchaseAPI.createPurchase $\rightarrow$ POST /api/fuel/purchases $\rightarrow$ FuelPurchaseController $\rightarrow$ FuelPurchaseService $\rightarrow$ FuelInventory.completeImmediatePurchase $\rightarrow$ FuelPurchaseRepository + FuelPurchaseDetailRepository $\rightarrow$ MySQL

DETALLE DEL FLUJO DE PRUEBA

No.	Acción / Entrada	Resultado Esperado
1	Usuario accede a /store, selecciona estación y combustible DIESEL, cantidad 50L	Formulario de compra con precio calculado
2	Frontend envía POST /api/fuel/purchases con { stationId, items:[{fuelType:"DIESEL", quantityLiters:50}] }	Request con JWT en header
3	FuelPurchaseService verifica stock disponible vía FuelInventory.canPurchase(50)	Retorna true si netAvailableStock $\geq$ 50
4	FuelInventory.completeImmediatePurchase(50) ejecutado	stockAvailable -= 50; stockTotal recalculado
5	FuelPurchase persistida con status COMPLETED; FuelPurchaseDetail por ítem	Registros en fuel_purchases y fuel_purchase_details
6	HTTP 201 retornado con FuelPurchaseDTO incluyendo totalAmount calculado	Total = 50 * priceCop (DIESEL)
7	Intento de compra con cantidad mayor al stock disponible	HTTP 400 "Stock insuficiente"

Resultado Esperado General: Compra procesada, stock descontado, registro persistido. Validación de stock insuficiente activa.

Criterio de Aceptación: HTTP 201, stock en BD decrementado, compra visible en /api/fuel/purchases/my.

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Estado: APROBADO	Responsable: Jonathan cobos y Camilo Barbosa
-------------------------	--

CASO DE INTEGRACIÓN CPI-015 — INTEGRACIÓN GOOGLE MAPS		
ID	CPI-015	
Caso de Uso	UCC-15 Cálculo de ruta hacia estación	
Objetivo	Validar integración entre el frontend React y la API de Google Maps para renderizado de mapa y cálculo de ruta.	
Precondiciones	API Key de Google Maps configurada. Estación con coordenadas registradas. Conexión a internet activa.	
Módulos Integrados	Map.jsx / Routes.jsx / MapView.jsx → @react-google-maps/api → Google Maps JavaScript API → Directions Service	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Usuario autenticado accede a /map	Mapa de Google renderizado con marcadores de estaciones
2	Frontend llama GET /api/stations/nearby?lat=4.67&lng=-74.12&radius=5	Lista de estaciones en rango; marcadores en mapa
3	Usuario selecciona una estación y solicita ruta	DirectionsService de Google recibe origen (geolocalización) y destino (coordenadas estación)
4	API Google retorna objeto DirectionsResult con legs y duration	Ruta trazada en mapa, tiempo y distancia mostrados
5	API Key inválida o cuota excedida	Error de Google Maps; NO se muestra ruta; mensaje de error al usuario
Resultado Esperado General: Ruta calculada y visualizada cuando API Key es válida. Caso de error de API externa detectado: el backend no maneja el fallo de Google Maps ya que es una integración directa frontend-API externa.		
Criterio de Aceptación: Ruta visible en mapa con API Key válida. Error visible al usuario con API Key inválida.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

CASO DE INTEGRACIÓN CPI-018 — DISPONIBILIDAD DE BASE DE DATOS	
ID	CPI-018

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Caso de Uso	Tolerancia a fallo de base de datos	
Objetivo	Verificar el comportamiento del sistema cuando MySQL no está disponible.	
Precondiciones	Backend Spring Boot activo. MySQL detenido intencionalmente.	
Módulos Integrados	StationController → StationService → StationRepository → MySQL (CAÍDO) → GlobalExceptionHandler	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	MySQL Server detenido	Servicio de BD no accesible
2	GET /api/stations/nearby?lat=4.67&lng=-74.12 enviado	Backend intenta conexión a MySQL
3	HikariCP agota el pool de conexiones	SQLException o CannotGetJdbcConnectionException lanzada
4	GlobalExceptionHandler captura RuntimeException	HTTP 400 retornado en lugar de HTTP 500 — COMPORTAMIENTO INCORRECTO
5	Frontend recibe error	Mensaje genérico al usuario — sin detalle del problema de BD
Resultado Esperado General: RECHAZADO: El GlobalExceptionHandler captura RuntimeException y devuelve HTTP 400 para errores de conexión a BD que deberían retornar HTTP 500 o 503. No existe manejo específico de DataAccessException.		
Criterio de Aceptación: HTTP 503 Service Unavailable con mensaje apropiado cuando BD no está disponible.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

4.Resultados de Pruebas de Aceptación (CPA)

Todos los casos de aceptación con usuario final resultaron APROBADOS. Los flujos principales del sistema satisfacen las necesidades del usuario final desde la perspectiva funcional y de experiencia de uso.

ID	Escenario	Actor	Criterio Clave	Estado
CPA-001	Registro y acceso al sistema	Usuario nuevo	< 5 seg, redirección /map	APROBADO
CPA-002	Búsqueda de estaciones cercanas	Conductor autenticado	Mapa < 3 seg, radio 8 km	APROBADO

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

CPA-003	Comparación de precios entre estaciones	Conductor autenticado	Precios 2 decimales COP, Recharts OK	APROBADO
CPA-004	Gestión de estación (Operador)	Usuario OPERATOR	Precio actualizado < 2 seg	APROBADO
CPA-005	Administración global de estaciones	Usuario ADMIN	CRUD completo, acceso 403 para USER	APROBADO
CPA-006	Compra de combustible por usuario	Usuario autenticado	Compra < 3 seg, historial actualizado	APROBADO

CASO DE ACEPTACIÓN CPA-001 — REGISTRO Y ACCESO AL SISTEMA		
ID	CPA-001	
Caso de Uso	El conductor desea crear una cuenta para usar DieselMaps	
Objetivo	Verificar que un usuario nuevo puede registrarse con sus datos personales y acceder al mapa de estaciones.	
Precondiciones	Sistema disponible. Navegador Google Chrome. Sin cuenta previa.	
Módulos Integrados	Register.jsx → Login.jsx → Map.jsx	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Abrir http://localhost:5175 y hacer clic en Registrarse	Formulario de registro visible con campos: nombre de usuario, correo, contraseña, teléfono, fecha nacimiento, rol
2	Ingresar datos válidos y seleccionar rol "Usuario"	Campos aceptan la información correctamente
3	Hacer clic en Crear Cuenta	Registro exitoso; redirección automática a /map con mapa de Google visible
4	Intentar registrarse con el mismo correo	Mensaje de error claro: "El correo ya está registrado"
5	Cerrar sesión y volver a iniciar con las credenciales registradas	Login exitoso; acceso al mapa restaurado
Resultado Esperado General: El usuario puede crear cuenta y acceder al sistema sin fricciones. Mensajes de error comprensibles. Criterio de Aceptación: Registro exitoso en menos de 5 segundos. Mensajes de error visibles. Redirección automática tras registro y login.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

CASO DE ACEPTACIÓN CPA-002 — BÚSQUEDA DE ESTACIONES CERCANAS		
ID	CPA-002	
Caso de Uso	El conductor desea encontrar estaciones de combustible cercanas a su ubicación	
Objetivo	Validar que el mapa muestra estaciones con sus precios y permite navegación hacia ellas.	
Precondiciones	Usuario autenticado. Geolocalización del navegador habilitada. Al menos una estación registrada en el sistema.	
Módulos Integrados	Map.jsx → useGeolocation.js → stationsAPI.getNearby → MapView.jsx → StationCard.jsx	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Usuario autenticado accede a /map	Mapa de Google centrado en ubicación del usuario
2	Sistema obtiene coordenadas del navegador y consulta GET /api/stations/nearby	Marcadores de estaciones aparecen en el mapa en radio configurado (default 8 km)
3	Usuario hace clic en un marcador de estación	StationCard muestra nombre, marca, precios por tipo de combustible
4	Usuario hace clic en "Ver detalles"	Redirección a /station/:id con información completa
5	Usuario no da permiso de geolocalización	Sistema muestra mensaje explicativo; usuario puede ingresar coordenadas manualmente o explorar el mapa
Resultado Esperado General: Estaciones cercanas visibles en mapa con precios. Navegación fluida entre lista y detalle.		
Criterio de Aceptación: Mapa carga en menos de 3 segundos. Estaciones dentro del radio visible. Precios mostrados correctamente.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

CASO DE ACEPTACIÓN CPA-003 — COMPARACIÓN DE PRECIOS	
ID	CPA-003
Caso de Uso	El conductor desea comparar precios entre varias estaciones

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Objetivo	Verificar que el módulo de comparación muestra precios actualizados y permite tomar decisiones informadas.	
Precondiciones	Usuario autenticado. Al menos 2 estaciones con precios diferentes registradas.	
Módulos Integrados	Compare.jsx → stationsAPI.getNearby → PriceTag.jsx → PriceHistoryChart.jsx (Recharts)	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Usuario accede a /compare	Página de comparación cargada con selector de estaciones
2	Selecciona 2-3 estaciones de la lista disponible	Tabla comparativa de precios por tipo de combustible (CORRIENTE, EXTRA, DIESEL, GAS)
3	Observa tendencias de precios	Gráfica de historial renderizada con Recharts; variaciones visibles en el tiempo
4	Identifica la estación con precio más bajo en DIESEL	Indicador visual resaltando el precio mínimo
Resultado Esperado General: Comparación funcional y visual. Datos de precios coherentes con los almacenados en BD. Criterio de Aceptación: Precios mostrados con exactitud (2 decimales COP). Gráfica Recharts renderizada. Diferencias de precio visibles.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

CASO DE ACEPTACIÓN CPA-004 — GESTIÓN DE ESTACIÓN (OPERADOR)		
ID	CPA-004	
Caso de Uso	El operador desea registrar y gestionar su estación de combustible	
Objetivo	Validar el flujo completo de gestión de estación desde la perspectiva del operador.	
Precondiciones	Usuario con rol OPERATOR autenticado. Ruta /create-station accesible (ProtectedRoute verifica rol).	
Módulos Integrados	CreateStation.jsx → stationsAPI.create → AdminStations.jsx → adminStationsAPI	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Operador accede a /create-station	Formulario de registro de estación visible

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

2	Completa nombre, marca (ej. Terpel), dirección, coordenadas y precios iniciales	Datos ingresados; mapa de selección de coordenadas funcional
3	Envía el formulario	Estación creada; confirmación visible; estación aparece en /map
4	Actualiza precio de DIESEL mediante el panel de la estación	Precio actualizado; historial registrado; alertas generadas para usuarios con favorito
5	Ajusta inventario de combustible (restock)	Stock incrementado; historial de inventario registrado
Resultado Esperado General: Operador puede gestionar su estación completamente, actualizaciones reflejadas inmediatamente en el sistema. Criterio de Aceptación: Estación visible para usuarios en /map. Precio actualizado en menos de 2 segundos. Historial de cambios.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

CASO DE ACEPTACIÓN CPA-005 — ADMINISTRACIÓN GLOBAL (ADMIN)		
ID	CPA-005	
Caso de Uso	El administrador desea gestionar todas las estaciones del sistema	
Objetivo	Verificar que el ADMIN puede activar, desactivar, editar precios e inventario de cualquier estación.	
Precondiciones	Usuario con rol ADMIN autenticado. Al menos 2 estaciones registradas.	
Módulos Integrados	AdminStations.jsx → adminStationsAPI → AdminStationController (@PreAuthorize ADMIN)	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Admin accede a /admin/stations	Lista completa de todas las estaciones del sistema
2	Desactiva una estación	Estación marcada como unavailable; desaparece del mapa para usuarios
3	Reactiva la estación	Estación vuelve a aparecer en el mapa
4	Actualiza precio de CORRIENTE en una estación	Precio actualizado; historial registrado
5	Actualiza inventario de GAS	Stock actualizado; historial reflejado

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

6	Usuario con rol USER intenta acceder a /admin/stations	ProtectedRoute redirige; HTTP 403 en API
Resultado Esperado General: Admin tiene control total del sistema, protección de rutas por rol funcional. Criterio de Aceptación: CRUD de estaciones por ADMIN exitoso. Acceso denegado a otros roles y cambios persistidos en BD.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

CASO DE ACEPTACIÓN CPA-006 — COMPRA DE COMBUSTIBLE (USUARIO)		
ID	CPA-006	
Caso de Uso	El conductor desea comprar combustible a través de la app	
Objetivo	Validar el proceso de compra desde la perspectiva del usuario final en la tienda.	
Precondiciones	Usuario autenticado. Estación con stock CORRIENTE ≥ 30L. Precio de CORRIENTE registrado.	
Módulos Integrados	Store.jsx → fuelPurchaseAPI.createPurchase → POST /api/fuel/purchases	
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
1	Usuario accede a /store	Tienda cargada con estaciones disponibles
2	Selecciona estación, elige tipo CORRIENTE, ingresa 30 litros	Total calculado automáticamente: 30 × precio CORRIENTE
3	Confirma la compra	Compra procesada; resumen con total en COP mostrado
4	Revisa historial de compras	Compra reciente visible en historial con fecha, estación y total
5	Intenta comprar más litros que el stock disponible	Mensaje de error claro: "Stock insuficiente"
Resultado Esperado General: Proceso de compra fluido. Total calculado correctamente. Error de stock insuficiente informado al usuario. Criterio de Aceptación: Compra completada en menos de 3 segundos. Total correcto. Historial actualizado. Mensaje de error para stock insuficiente.		
Estado: APROBADO		Responsable: Jonathan cobos y Camilo Barbosa

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

5. Resultados de Pruebas de Sistema (CPS)

ID	Escenario del Sistema	Verificación	Estado
CPS-001	Flujo completo usuario nuevo: registro → login → mapa → favorito → alerta	Favorito guardado, alerta generada al cambiar precio	APROBADO
CPS-002	Flujo operador: login → crear estación → actualizar precio → ver historial	Estación en mapa, historial en BD, alerta a favoritos	APROBADO
CPS-003	Flujo admin: desactivar → reactivar estación	Estado consistente en BD; estación desaparece y reaparece del mapa	APROBADO
CPS-004	Flujo compra: stock decrementado y historial actualizado	Stock 50→20L; compra en historial; total correcto	APROBADO
CPS-005	Control de sesión: token JWT expirado → redirección automática	Interceptor 401 activo; localStorage limpiado; redirige a /login	APROBADO
CPS-006	Acceso a rutas protegidas sin autenticación	ProtectedRoute redirige; API retorna 401	APROBADO
CPS-007	Consistencia: actualización de precio refleja en historial y alertas	PriceHistory creado; Alert generada; FuelPrice.prevPriceCop actualizado	APROBADO
CPS-008	Manejo de error global: endpoint inexistente	HTTP 404/400; no expone stack trace (parcialmente cubierto)	APROBADO
CPS-009	Estabilidad: 5 usuarios concurrentes consultando nearby	Sin deadlocks; respuestas < 1s; todas correctas	APROBADO
CPS-010	Validación de campos obligatorios en todos los formularios	HTTP 400 retornado; frontend no muestra todos los errores (parcial)	APROBADO

6. Pruebas de Atributos de Calidad

6.1 Pruebas de Carga — Apache JMeter

Se diseñaron tres escenarios de carga progresivos sobre los endpoints críticos del backend. Los umbrales de aceptación se establecieron con base en el plan de pruebas (P95 < 2000 ms, tasa de error < 1% en escenario base).

Parámetro	Escenario Base	Escenario Medio	Escenario Pico	Umbral Aceptación
Usuarios concurrentes	10	50	100	—
Ramp-up (segundos)	30	60	90	—
Duración (segundos)	120	300	600	—

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Errores aceptables	< 1%	< 2%	< 5%	< 1% base
P95 tiempo de respuesta	< 2000 ms	< 2000 ms	< 5000 ms	< 2000 ms

Endpoints Críticos Identificados para Pruebas de Carga

- GET /api/stations/nearby — CRÍTICO: usa haversine en SQL; cuello de botella potencial con full table scan
- POST /api/auth/login — ALTO: BCrypt.matches es costoso computacionalmente bajo concurrencia alta
- POST /api/fuel/purchases — ALTO: transacción con bloqueo de inventario; riesgo de race condition
- GET /api/stations/{id} — ALTO: consulta frecuente con relación lazy FuelPrice
- PATCH /api/stations/{id}/prices — MEDIO: escritura en PriceHistory y AlertRepository en cascada

6.2 Análisis de Desempeño

Componente	Riesgo	Causa Técnica
StationRepository.findNearby (haversine)	ALTO	Fórmula haversine en SQL sin índice espacial
BCryptPasswordEncoder.matches en login	MEDIO	BCrypt strength=10 por defecto; intencionalmente lento (seguridad)
StationService.updatePrice con alertas en cascada	MEDIO	Loop sobre todos los usuarios con favorito; AlertRepository.save() uno a uno
GET /api/admin/stations (findAll sin paginación)	MEDIO	Retorna TODAS las estaciones sin Pageable; OutOfMemory con datos grandes

6.3 Evaluación de Escalabilidad

La arquitectura monolítica (Spring Boot + MySQL en servidor único) tiene capacidad limitada de escala horizontal. Proyección basada en análisis de código:

Métrica	50 usuarios	200 usuarios	500 usuarios	Acción > 200
Tiempo respuesta /nearby	< 300 ms	< 800 ms	> 2000 ms	Caché Redis + índice
Throughput estimado	~80 req/s	~40 req/s	~15 req/s	Load balancer
Conexiones MySQL activas	5–10	20–40	> 50 (pool agotado)	Aumentar pool HikariCP
Memoria JVM Heap	~150 MB	~300 MB	~600 MB+	Ajustar -Xmx512m+

	Informe de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

6.4 Evaluación de Disponibilidad

Escenario	Comportamiento Actual	Comportamiento Esperado	Estado
Backend caído	Error genérico de Axios	Mensaje amigable; opción reintentar	DEFICIENTE
Google Maps API no disponible	Mapa en blanco; sin mensaje	Mensaje explicativo; funcionalidad sin mapa	DEFICIENTE
Token JWT expirado	Interceptor Axios detecta 401; redirige a /login	Comportamiento correcto implementado	ADECUADO
Reinicio del backend	Sesiones JWT persisten (stateless)	Usuarios no pierden sesión	ADECUADO

6.5 Análisis de Modificabilidad — SonarQube

Resultados proyectados del análisis de código fuente con SonarQube 10.x Community:

Métrica SonarQube	Rating / Valor	Observación
Reliability Rating	A (objetivo)	GlobalExceptionHandler retorna 400 para errores de BD — bug de clasificación HTTP
Security Rating	B (mínimo)	JWT secret hardcodeado en application.properties — Security Hotspot confirmado
Maintainability Rating	A–B (objetivo)	StationService con 7+ dependencias inyectadas (responsabilidad múltiple)
Technical Debt	< 2 horas	Lógica anidada en FuelPurchaseService; patrón de userId repetido en 2 métodos
Duplications	< 5%	findByUsername().getId() repetido en FuelPurchaseController — extraer método privado
Code Smells estimados	< 20	UserController con @CrossOrigin duplicando CORS; FuelInventory con lógica de negocio en entidad
Cobertura JUnit	> 60% (objetivo)	Tests existentes: AuthServiceTest, StationServiceTest, UserServiceTest, JwtUtilTest. Faltan: FuelInventoryService, FuelPurchaseService, AdminStationService